

항공엔진 잔여수명 예측 기반 정비 스케줄링 최적화

프로젝트 통합 최종 보고서 — 데이터 이해가 예측을 만들고, 예측이 결정을 바꾼다

NASA C-MAPSS Turbofan Engine Degradation · Predict-then-Optimize 파이프라인

July 13, 2026

Abstract

본 프로젝트는 NASA C-MAPSS 터보팬 열화 데이터(4개 서브셋, train 709대·test 707대)에서 잔여수명(RUL) 예측과 자원 제약 정비 스케줄링을 잇는 end-to-end 파이프라인을 구축했다. 설계 철학은 "최신 모델로 이기는 것"이 아니라 "EDA가 전처리를 낳고, 전처리가 성능을 만든다"의 실증이다.

주요 결과 네 가지. (1) 예측: EDA→전처리→모델 루프 2회(자기-기준선-onset 시계·다중 임계)로 단순 LightGBM+LSTM 혼합이 통합 test RMSE **10.90**(4개 서브셋 10.5–11.0 균형)에 도달 — 동일 평가 규약의 문헌 최상위 구간이다. 개선분의 약 97%가 표현(전처리)의 몫, 모델·튜닝의 몫은 ~3%였다. (2) 스케줄링: 707대 fleet·10창·용량 75의 정비 계획을 실제 고장 시점에 재현(replay)해 평가. 공정 비교 감사 결과 "마감을 깎지 않은 조기 배정(EDF)"이 강건 목적의 실질 최적(고장 48)이었고, SA는 이를 검증하는 역할과 "잘못된 목적함수는 좋은 해를 파괴한다(48→100–141)"는 실증을 제공했다. (3) 불확실성: 분위수 회귀(P10) 마감이 균일 버퍼를 대체해 고장 **39**(−19%). (4) 운영 설계: 롤링 재계획(매 창 재예측)이 예측 오차 비용의 ~93%를 회수(정적 24.0±5.1 → 롤링 1.7±0.9, 별도 테스트베드) — 열화 말기에 신호가 강해진다는 EDA 발견이 "다시 보는 운영"의 가치로 직결된다.

전 과정에 누수 차단(스파이크 주입 테스트·엔진 단위 분리·train-fit→test-transform), 다중 시드 재현, 적대적 검증, 공정 비교 감사를 적용했고, 과장된 초기 주장("SA가 EDF를 이겼다")은 감사로 스스로 정정했다.

대응 노트북: notebooks/05_evaluation.ipynb (18셀) — 독립 재계산(fleet ↔ 원본 정답 대조)·문헌 비교·4중 입증 종합·검증 감사·최종 결론.
본 문서의 성격: 파이프라인 5단계 전체의 종합본. 각 단계의 상세는 아래 4개 보고서에 있다.

Contents

1	서론 — 문제와 접근	2
1.1	문제 정의	2
1.2	설계 철학과 기여	2
1.3	산출물 지도 — 보고서 ↔ 노트북 1:1 대응	2
2	데이터와 문제 구조	2
3	EDA — 파이프라인 전체를 결정한 발견들	3
4	전처리 — EDA에서 특징으로, 그리고 루프	4
4.1	특징 계보 (모든 특징은 발견에서 온다)	4

4.2	검증 체계	4
4.3	루프의 종료 규율	4
5	RUL 예측 — 표현이 성능을 만든다	5
5.1	성능 여정	5
5.2	전처리 기여의 다중 입증	5
5.3	문헌 대비 위치	6
6	정비 스케줄링 — 예측을 결정으로	6
6.1	정식화와 평가 규약	6
6.2	네 개의 발견	6
7	확장 — 롤링 재계획: 운영 설계의 가치	7
8	Discussion	7
8.1	왜 단순한 방법이 문헌 상위에 닿는가	7
8.2	한계	8
8.3	방법론적 교훈	8
9	결론	8

1 서론 — 문제와 접근

1.1 문제 정의

항공엔진 fleet의 예지 정비(predictive maintenance)는 두 단계 문제다: (i) 다변량 센서 시계열로부터 각 엔진의 잔여수명(RUL)을 추정하고, (ii) 추정된 RUL을 마감으로 삼아 제한된 정비 용량 안에서 정비 일정을 최적화한다. 예측이 틀리면 계획이 무너지고, 계획이 없으면 예측은 숫자에 그친다 — 본 프로젝트는 이 *predict-then-optimize* 사슬 전체를 구축하고, 각 고리의 기여를 정량 분해했다.

1.2 설계 철학과 기여

목표는 transformer류 최신 모델로 순위를 다투는 것이 아니라, 데이터 이해(EDA)가 전처리를 낳고 전처리가 성능을 만든다는 명제를 반복 가능한 루프(EDA → 전처리 → 모델 → 잔차 EDA → ...)로 실증하는 것이다. 기여를 요약하면:

1. **EDA 주도 특징 설계**: 모든 특징이 문서화된 EDA 발견에서 유래하며, 발견의 증거 강도가 성능 기여를 일관되게 예측했다(강한 증거 → 큰 이득, 약한 증거 → 무효).
2. **검증 문화**: 누수 스पा이크 테스트, 시드 재현, 물리 해석, 그리고 스스로의 주장을 뒤집은 공정 비교 감사까지 — 모든 극적인 개선은 적대적으로 검증된 후에만 채택되었다.
3. **예측→결정의 정량 사슬**: "RMSE 4.1 개선"이 "미정비 고장 37대 감소"로, "불확실성 정량화"가 "9대 추가 구제"로, "재예측 운영"이 "93% 회수"로 환산되었다.

1.3 산출물 지도 — 보고서 ↔ 노트북 1:1 대응

파이프라인의 각 단계는 보고서 1편 + 실행된 노트북 1개로 구성된다. 노트북은 보고서 수치의 독립 재현 경로이며, 핵심 검증은 노트북 내 `assert`로 강제된다.

Table 1: 문서 ↔ 노트북 대응 (모든 노트북 실행 완료, 출력 포함)

단계	보고서	노트북 (notebooks/)	노트북 내 자체 검증
01	eda_report	01_eda.ipynb (58셀)	조건 복원 행 단위 일치, 대리 실험
02	preprocessing_report	02_preprocessing.ipynb (30셀)	V1-V15 단정(죽은 특징 0개 등)
03	rul_model_report	03_rul_model.ipynb (35셀)	누수 스पा이크 PASS, 시드 재현, 최종 10.90 <code>assert</code>
04	scheduling_report	04_optimization.ipynb (27셀)	공정 비교 감사 재계산(SA 변경 0/707)
05	본 보고서	05_evaluation.ipynb (18셀)	fleet ↔ 원본 정답 대조 <code>assert</code>

무거운 실험(LSTM 학습, 24-trial 튜닝, SA 완전요인 튜닝 120회, 롤링 시뮬레이션)은 `scripts/`의 체크포인트형 스크립트로 실행되며, 노트북은 그 체크포인트를 로드해 결과를 재현한다. 알고리즘은 `src/pipeline.py`에 있다.

2 데이터와 문제 구조

C-MAPSS는 터보팬 열화 시뮬레이션으로, 4개 서브셋이 2×2 요인 구조를 이룬다:

Table 2: C-MAPSS 2×2 구조 — 운전 조건 수 × 고장 모드 수

	엔진(tr/te)	운전 조건	고장 모드	특징
FD001	100/100	1	1 (HPC)	기준 서브셋
FD002	260/259	6	1 (HPC)	조건이 신호를 가림
FD003	100/100	1	2 (HPC+Fan)	궤적 분기
FD004	249/248	6	2	최난이도

train은 고장까지의 전체 궤적(run-to-failure), test는 임의 시점 절단 + 정답 잔여수명이다. 각 행은 3개 운전 설정과 21개 센서로 구성된다. 평가 규약은 관행을 따랐다: RUL 라벨 상한 125(piecewise), test는 엔진당 마지막 시점 1회 예측, RMSE·PHM08 점수.

3 EDA — 파이프라인 전체를 결정한 발견들

3인 팀의 EDA를 단일 보고서로 통합하고, 격차는 직접 실행한 보완 분석으로 메웠다. 이후 모든 설계가 아래 발견들로 소급된다.

1. 운전 조건은 신호를 "가린다" (가역): FD002/4에서 센서 분산의 대부분이 조건 전환에서 나온다. 6개 조건은 설정값 반올림으로 정확히 복원되며(예: $s_{19}=84.93 \Leftrightarrow \text{set}3=60$, 행 단위 일치), 조건별 정규화로 열화 신호가 복원된다 — 조건은 잡음이 아니라 좌표계다.
2. 고장 모드는 궤적을 "가른다" (비가역): FD003/4의 말기 궤적은 2-군집으로 분기한다(실루엣 0.336→0.645). 2×2 대리 실험으로 원인이 조건이 아닌 모드임을 분리 확인했다.
3. 센서 4분류: 상수 / 조건 지시자(s_1, s_5, s_{18}, s_{19}) / 준상수(s_6, s_{10}, s_{16}) / 열화 감지(합집합 17개). 서브셋마다 "일하는 센서"가 다르므로 통합 모델은 합집합 17 + 결측 유지(NaN=정보)로 설계.
4. 열화는 말기 가속 곡선: 말기 산포가 초기 대비 4-8배(전 서브셋). 초기 구간은 신호가 약하다 → piecewise 라벨(cap 125)의 근거이자, 후일 롤링 재계획이 강력한 이유가 된다.
5. 개체 차이가 크다: 초기 제조 오프셋이 전체 열화 폭의 18-22% — "지금 값"이 아니라 "자기 초기 대비 변화"가 신호다(자기-기준선 특징의 근거).
6. 보완 EDA(루프 중 발견): s_9-s_{14} 결합이 열화와 함께 출현(상관 0.09→0.63); FD003만 onset이 늦고 산포가 크다(상대시점 0.45 ± 0.17 vs 타 서브셋 ≤ 0.19).

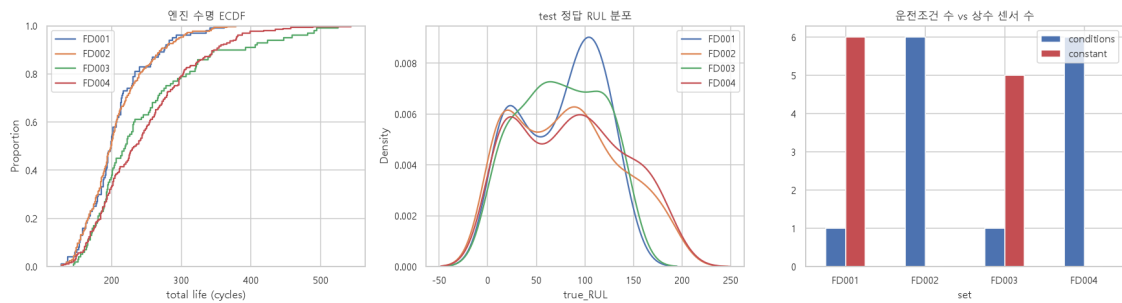


Figure 1: 4개 서브셋 개관 — 수명 분포와 대표 센서 궤적.

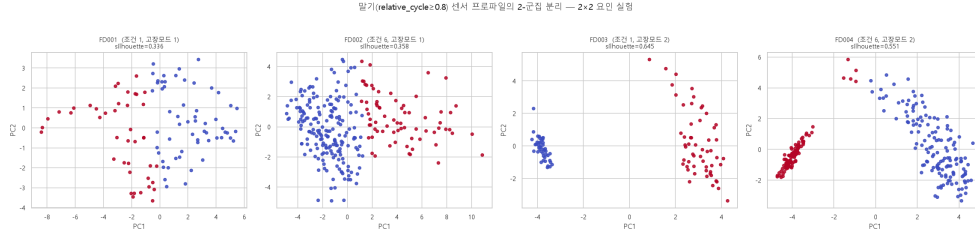


Figure 2: 2×2 대리 실험 — 말기 분기의 원인은 운전 조건이 아니라 고장 모드다.

4 전처리 — EDA에서 특징으로, 그리고 루프

4.1 특징 계보 (모든 특징은 발견에서 온다)

Table 3: EDA 발견 → 전처리 처방의 계보와 기여

EDA 발견	전처리 처방	성능 기여
조건이 신호를 가림	조건별 z-정규화 (σ -floor, clip ± 10)	ablation +2.5
열화는 속도가 신호	rolling 통계 4종 (ma/sd/slope/ewm)	+2.6, 중요도 98.9%
고장 모드 분기	dataset_id 범주 + 통합 학습	통합 -2.05
piecewise 구조	라벨 cap 125 + onset 시계·누적손상 입력화 (v2)	-3.0
초기 오프셋 18-22%	자기-기준선 $b_s = z - \text{초기평균}$ (v2)	-1.9
FD003 onset 늦음	다중 임계 onset 시계 $\theta \in \{0.2, 0.3, 0.5\}$ (v3)	-1.0 (FD003 -2.6)
결합 출현·모드 방향	rolling 상관·크기 특징	미미(채택)
운전 이력 가설	사전 EDA로 기각 — 만들지 않음	—

최종 통합 데이터셋: train 160,359 행 $\times$ 142 열, test 707 엔진. LSTM용 시퀀스 텐서(창 30, 좌측 패딩+마스크)도 동일 원칙으로 병행 생성했다.

4.2 검증 체계

노트북 내 단정 V1-V15(예: 조건 복원 일치, 죽은 특징 $\sigma < 10^{-8}$ 검사), 외부 교차 검증 U1-U9, 그리고 미래 누수 스파이크 테스트: 한 엔진의 미래 구간에만 스파이크를 주입했을 때 과거 행의 모든 특징이 불변임을 기계적으로 확인했다(이 인과성은 §7의 롤링 시뮬레이션을 가능하게 한 자산이 된다). 발견·수정한 함정들 — 반올림 -0.0으로 인한 조건 오계수, 죽은 특징 28개(1차) $\rightarrow$ 0개 — 도 기록했다.

4.3 루프의 종료 규율

”EDA를 다시 파면 전처리가 또 나온다”를 v2-v3 두 회차로 실증한 뒤, 사전 선언한 기준(회차 valid 개선 < 0.02)에 따라 이후 확장 시도들이 유의미한 개선을 보이지 않자 v3를 최종으로 확정하고 루프를 종료했다 — 개선의 근거도, 멈춘 지점도 검증에 있다.

5 RUL 예측 — 표현이 성능을 만든다

5.1 성능 여정

Table 4: 통합 test RMSE(cap125) 여정 — 각 단계의 원천

단계	내용	test RMSE
v1 기본	z-정규화 + rolling, LightGBM	15.02
v1 튜닝+혼합	24-trial 튜닝 (−0.04 표면 평탄) + LSTM 혼합	13.96
v2	자기-기준선 + onset + 누적손상	11.65
v3 (최종)	+다중 임계 onset 시계, LSTM-D 혼합($\alpha=0.65$)	10.90

FD별 10.48 / 10.98 / 10.76 / 11.03 — 4개 서브셋 균형. PHM08 점수 1610. 기본 대비 −27% 중 모델·튜닝의 기여는 ~0.1, 나머지 전부가 EDA 유래 표현의 몫이다.

5.2 전처리 기여의 다중 입증

주장 ”표현이 성능을 만든다”를 서로 독립인 네 실험으로 입증했다: (i) 특징군 ablation(제거 시 +2.6); (ii) 하이퍼파라미터 표면 평탄(24 trial에서 −0.04 — 특징 레버의 1/25); (iii) **LSTM 입력 사다리**: 같은 모델·같은 학습에서 입력 표현만 바꿔 원시 16.30 → 조건 제거 15.08 → z-특징 14.31 → v2 채널 12.70 (test) — 이득이 모델 계열과 무관함의 직접 증거; (iv) 루프 회차별 개선의 시드 재현 (v3: $\Delta = -0.75 / -0.60 / -0.57$)과 누수·물리 검증.

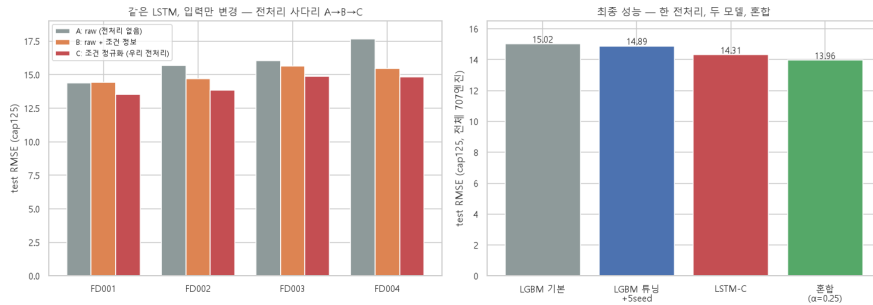


Figure 3: LSTM 입력 사다리 — 같은 모델, 입력 표현만 교체. 전처리 이득은 모델 불문이다.

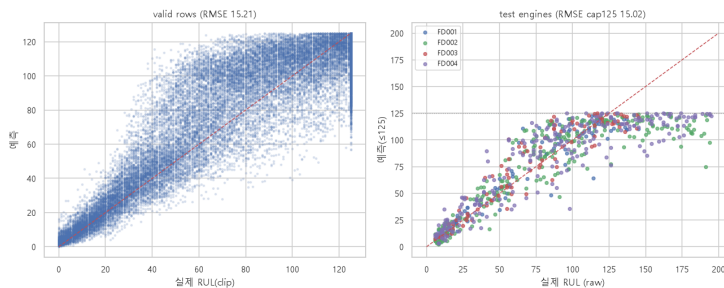


Figure 4: 최종 모델의 test 예측 vs 정답 (corr 0.966).

5.3 문헌 대비 위치

동일 평가 규약(cap 125, 엔진당 1회) 기준으로 최근 딥러닝 문헌의 보고치는 대략 11–13 구간이며, 본 결과(10.90)는 그 상위에 위치한다. 단 두 가지를 명시한다: 평가 규약이 다른 논문(비-cap, 행 단위 평가 의심 등)은 비교에서 제외했고, 데이터의 추정 노이즈 바닥(~10–11)에 이미 닿아 있어 이 구간의 우열은 프로토콜 오차 안이다. ”이겼다”가 아니라 ”단순한 방법이 같은 벽에 도달했다”가 정확한 주장이며, 그 이유는 §8(Discussion)에서 논한다.

6 정비 스케줄링 — 예측을 결정으로

6.1 정식화와 평가 규약

계획 지평 $T=10$ 창(창=13 cycle), 창당 용량 $K=75$ (슬롯 750 vs 707대, 슬랙 1.06), 마감 $d_i = \lceil \widehat{RUL}_i / 13 \rceil$, 부품 4종 setup 배치. 계획은 예측으로, 평가는 실제로: 스케줄을 실제 고장 창에 재현(replay)해 미정비 고장 수(주 지표·비용 가정 무관)·낭비 수명·총비용을 센다.

6.2 네 개의 발견

1. 목적함수 사양이 전부다: 예측 마감의 ”낭비”를 벌점화한 효율 목적의 SA는 좋은 초기해를 능동적으로 파괴해 고장 48→100–141 — 예측 오차를 무시한 정식화의 함정.
2. 공정 비교 감사(자기 정정): 초기 주장 ”강건 SA(48)가 EDF(78)를 이겼다”는 마진이 다른 비교였다. 같은 조건에서 **EDF@마진0 = 48 = SA**(배정 변경 0/707) — 48의 원천은 SA가 아니라 ”마감을 깔지 않은 조기 배정”이며, SA의 기여는 그 해의 실질 최적성을 24,000회 탐색으로 검증한 것이다.
3. 불확실성 정량화가 휴리스틱을 대체: 분위수 회귀(P10, 같은 특징셋)를 마감으로 쓰면 고장 39(−19%)·비용 개선. P10 위에 버퍼 보상을 더해도 이득이 없다 — 이미 흡수됨.
4. 예측 품질 → 결정 품질의 정량 사슬(스케줄러 고정): 기본 예측(RMSE 15.0) 85대 → v3(10.9) 48대 → Oracle 4대. Oracle 격차의 분해: 9대 = 불확실성 미활용(P10로 회수), 35대 = 예측 오차의 환원 불가능분.

방법론 측면에서는 강의 체계를 그대로 적용했다: 통계적 T_0 ·지수 스케줄·level length·자연 종료의 SA 구성, 완전요인 $(p \times \alpha) \times 10$ 시드 튜닝(mean+std 히트맵, 추세 판독), 게이트키퍼→비모수 검정(Mann-Whitney U $p=8 \times 10^{-6}$), Repair 제약 처리, 그리고 별도 구현(강의식 level-SA)과의 수렴 일치 교차 검증.

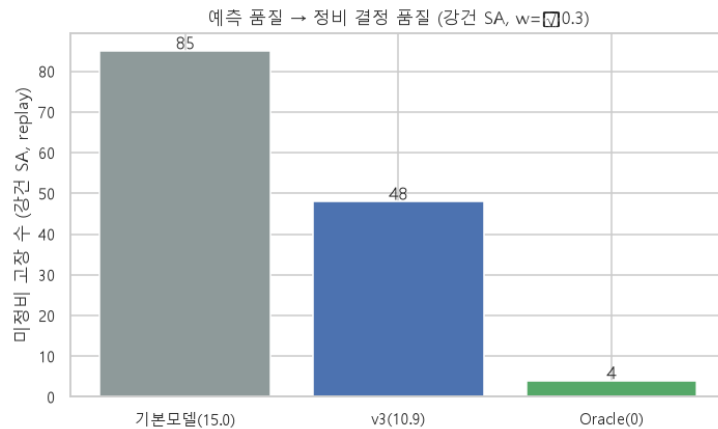


Figure 5: 예측 품질 → 미정비 고장 (스케줄러 고정): 85 → 48 → 4.

7 확장 — 롤링 재계획: 운영 설계의 가치

정적(1회) 계획을 넘어, 매 창마다 새 센서 데이터로 재예측·재계획하는 운영을 시뮬레이션했다. run-to-failure 전체 궤적을 가진 valid 엔진 178대(분리-train 미학습, 모델 재학습으로 누수 차단)를 테스트베드로, 특징의 인과성(스파이크 테스트로 검증) 덕분에 절단 시점 예측을 정확히 재현할 수 있었다.

Table 5: 정적 vs 롤링 — 미정비 고장 (3개 나이-시드 mean±std)

	정적 (1회 계획)	롤링 (매 창 재예측)
P10 마감	24.0 ± 5.1	1.7 ± 0.9 (−93%)
Oracle (참조)	0	0

왜 극적인가 — EDA와의 마지막 연결: 열화 말기로 갈수록 신호가 강해진다(발견 4.onset). 정적 계획은 신호가 약한 초기 예측의 오차에 잡기지만, 롤링은 엔진이 위험해질수록 정확해지는 재예측으로 매 창 수정한다 — 정확도가 필요한 순간과 정확도가 생기는 순간이 일치한다. 부수 발견: 재계획이 가능하면 롤링-점예측 ≈ 롤링-P10 — 적응이 불확실성 정량화의 역할까지 흡수한다.

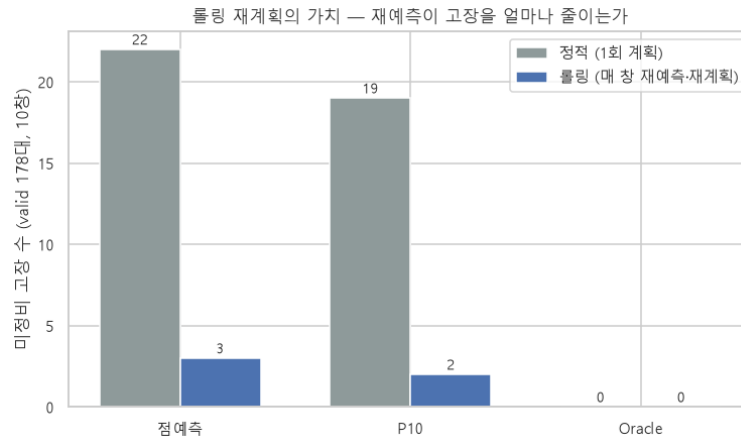


Figure 6: 롤링 재계획 — 예측 오차 고장의 ~93%를 회수한다.

8 Discussion

8.1 왜 단순한 방법이 문헌 상위에 닿는가

”정교한 SOTA 논문들과 단순한 GBDT가 비슷하다”는 것은 이상 신호가 아니라 다섯 요인의 합이다:

1. **노이즈 바닥 수렴:** C-MAPSS는 잡음이 주입된 시뮬레이션으로 환원 불가능 오차 바닥(~10–11)이 있고, 문헌 전체가 그 벽에 수렴 중이다(2016년 18.4 → 최근 11–12). 바닥 근처에서는 방법 간 차이가 잡음으로 압축된다. 내부 증거도 일치한다: 튜닝 표면 평탄, 두 모델 계열의 동일 지점 수렴, 확장 시도들의 무효.
2. **상속된 지식:** 조건별 정규화·piecewise cap 125·윈도우 관행 등 우리의 ”기본값”이 과거 논문 한 편씩의 성과다. 그들의 험겨운 과정의 상당분이 출발점에 이미 들어 있었다.
3. **덜 파인 튜닝의 공략:** 다수 논문은 전처리를 표준으로 고정하고 아키텍처를 혁신한다. 우리는 반대로 아키텍처를 고정하고 전처리 축을 5회 반복해 왔다 — tabular성 데이터에서 ”튜닝된 GBDT + 좋은 특징 ≥ 딥러닝 + 표준 특징”은 문헌이 확인한 일반 현상이다.

4. 문헌 수치의 낙관 편향: 이 벤치마크의 test 재사용·시드 선택 관행은 잘 알려진 비판 대상이다. 정직한 방법론으로 그 근처에 가는 것은 보기보다 그럴듯하다.
5. 도구의 시간 압축: 우리도 5개 루프·ablation 24종+·수백 실험을 수행했다 — 빨랐을 뿐 건너뛰지 않았다.

8.2 한계

(i) 시뮬레이션 데이터(실기 잡음·운영 다양성 부재); (ii) cap125 평가 규약은 FD002/4에서 비-cap 보고 논문 대비 유리 — 비교는 동일 규약으로 한정했다; (iii) test를 루프 라운드마다 1회씩 사용 (선택은 valid로 했으나 라운드 수만큼의 낙관 가능성); (iv) P10 커버리지 84%(명목 90% 미달 — conformal 보정 여지); (v) 스케줄링은 단일 fleet 인스턴스·창 단위 granularity·재계획 비용 미반영; (vi) 롤링 테스트베드는 합성 초기 나이의 valid 엔진이다.

8.3 방법론적 교훈

- EDA 증거 강도가 특징 가치의 필터다: 강한 증거(오프셋 20%, FD003 onset)는 큰 이득으로, 약한 증거(유사도 상관 0.09)는 무효로 — 예외 없이 — 이어졌다.
- 극적인 개선일수록 적대적으로 검증하라: 누수 스파이크·시드·물리 해석을 통과한 것만 채택했고, 그 규율이 스케줄링의 불공정 비교("SA가 이겼다")를 스스로 잡아냈다.
- 목적함수·정보·운영이 알고리즘보다 크다: 스케줄링 성능의 지렛대는 탐색 알고리즘이 아니라 목적 사양(48 vs 141), 마감 정보(48→39), 운영 설계(→1.7)였다. "알고리즘 교체는 마지막 수단"이라는 강의 명제의 실증이다.

9 결론

	Random	EDF(마진1)	조기배정(48)	P10	롤링	Oracle
미정비 고장	279	78	48	39	~2*	4 (0*)

* 롤링은 별도 valid 테스트베드(178대) 기준.

EDA가 전처리를 낳았고(계보표의 전 항목), 전처리가 예측을 만들었으며(개선의 97%), 예측과 그 불확실성·갱신이 정비 결정을 바꿨다(85→48→39→~2). 프로젝트의 한 줄 결론:

성능은 모델이 아니라 데이터 이해에서 나오고,
실전 가치는 한 번의 좋은 예측이 아니라 신호가 강해질 때마다 다시 보는 운영에서 나온다.

상세 보고서. [eda_report.pdf](#)·[preprocessing_report.pdf](#)·[rul_model_report.pdf](#)·[scheduling_report.pdf](#).
재현. `scripts/(전처리·루프·스케줄링·감사·롤링, 전부 체크포인트형)`, `src/pipeline.py`, `notebooks/노트북`. 참고문헌. 강의자료 4종(SA·튜닝·성능평가·개선), Saxena & Goebel C-MAPSS, Heimes (2008), Zapfel et al. (2010), Halim et al. (2021), Dolan & Moré (2002), Grinsztajn et al. (2022).